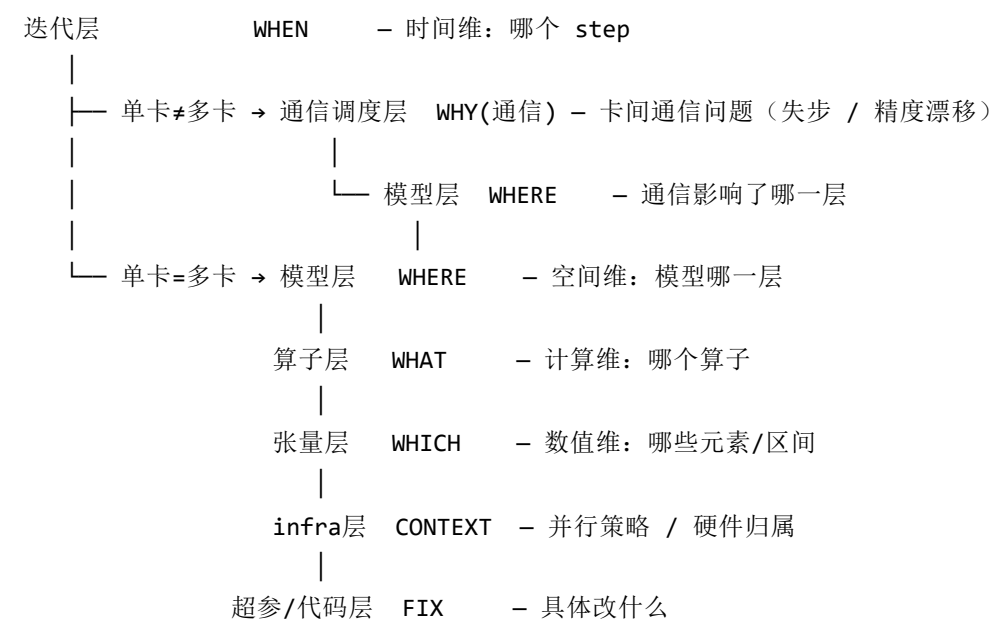


训练精度问题定位链

从表象精度异常出发，逐层下钻直至根因。**迭代层后即分叉**：单卡/多卡复现是否一致，决定走「通信分支」还是「计算主干」。每一层有明确的**判据**决定下一步方向。



1. 迭代层 — 锁定「哪个 step」

项目	内容
定位目标	精度异常首次出现（或突然恶化）的训练步
观测手段	loss 曲线、acc/PPL 曲线、grad_norm 曲线，按 global_step 展开
判据	loss spike / NaN / grad_norm 突变的 step 即为嫌疑步；若整条曲线平稳漂移，取漂移起始段
产出	嫌疑 step 列表（如 step 15320~15350）

⚠️ 此处产生**关键分叉**：用同一数据/种子在单卡和多卡环境各跑一次嫌疑 step。

- **单卡复现 = 多卡复现** → 计算问题，沿主干进入「模型层」
- **单卡复现 ≠ 多卡复现** → 通信问题，切入分支「通信调度层」

2. 模型层 — 锁定「哪一层」

项目	内容
定位目标	嫌疑 step 内，异常发生在 Emb / Attn / FFN / MoE 中的哪一层（或哪些层）
观测手段	层 × step 热力图：每层逐 step 的 grad_norm（反向追因）或激活值离群率
判据	热力图中对应 step 的纵向高亮带，定位到具体 layer index；Dense 层与 MoE 层分开标注
产出	问题层路径，如 model.layers.5.self_attn 或 model.layers.42.mlp

—— 分支：通信调度层 ——

从迭代层直接分叉。当单卡多次运行结果一致、多卡运行结果出现差异时，进入此分支。通信分支定位完毕后，仍需回到模型层确定通信问题影响了哪些层。

项目	内容
定位目标	判断通信问题类别：调度失步（all-reduce / all-to-all / p2p 时序错位）还是浮点累加舍入误差累积（精度对齐漂移）
判据	单卡多次结果一致 + 多卡结果不一致 → 通信问题；单卡多次结果不一致 → 非通信问题（可能是随机性/dropout/数据差异），回到主干
观测手段	NCCL Inspector 通信概览表（带宽/延迟异常）、通信算子 trace 时间线（看 send/recv 配对、barrier 等待）、各 rank 的 all-reduce/all-to-all 输入/输出 hash 对比
产出	通信异常类型 + 涉及的 rank 列表 + 通信原语（all-reduce / all-to-all / p2p）

3. 算子层 — 锁定「哪个算子」

项目	内容
定位目标	在问题层内，定位到具体的异常算子（如 q_lora 、 attn_scores 、 gate_proj 、 router ）
观测手段	层内算子误差瀑布图（横轴：按执行顺序排列的算子，纵轴：与 baseline 的误差），定位误差突变点
判据	误差在某个算子处出现量级跳跃（如 1e-6 → 1e-3），即为首害算子

项目	内容
产出	问题算子路径，如 <code>model.layers.5.self_attn.q_lora</code>

4. 张量层 — 锁定「哪些元素/数值区间」

项目	内容
定位目标	在问题算子的输入/输出中，定位到哪些元素位置、哪些数值区间引入了偏差
观测手段	dump 该算子的输入、激活、梯度张量，与 baseline（单卡正确结果或上一正常 step）逐元素对比；对 softmax、layernorm 等数值敏感算子，绘制散点图、直方图、TopK 误差表
判据	逐元素 diff 中绝对值最大的元素即为嫌疑元素；直方图中偏离 baseline 分布的区间即为嫌疑数值区间
产出	嫌疑元素索引 + 数值区间范围 + 偏差量级

5. infra层 — 锁定「并行策略 / 硬件」

项目	内容
定位目标	确定问题涉及的并行策略（PP/EP/TP/DP）和硬件范围（哪些 rank、哪些节点）
观测手段	从 trace 中提取问题算子涉及的并行策略标签（pp/ep rank）、通信原语、所属 GPU 节点；结合通信分支结论交叉定位
判据	问题算子仅出现在特定 pp stage 或 ep rank → 局部硬件/策略问题；跨所有 rank 出现 → 全局算法/精度问题
产出	涉及的 rank 列表 + 并行策略维度 + 嫌疑硬件范围（如 node3 GPU 0~7）

6. 超参/代码层 — 锁定「改什么」

项目	内容
定位目标	将根因映射到具体可执行的修改项
观测手段	回溯：启动参数（学习率、warmup、batch_size、precision）、模型代码（算子实现、数值精度 cast）、通信业务代码（通信原语调用、同步点）、训练超参（dropout rate、weight decay）
判据	根据上游各层锁定的根因类型匹配修改项：数值溢出 → 调低 lr / 开 loss scaling；通信失步 → 修复同步点；专家路由倾斜 → 调整 load balance 策略

项目	内容
产出	具体修改项 + 修改文件/参数路径 + 验证方案

案例一：通信分支 — MoE all-to-all 超时导致 loss NaN

路径：迭代层 → 单卡≠多卡 → 通信调度层 → 模型层 → infra层 → 超参/代码层

背景：64 GPU 训练 DeepSeek-V3.2，EP=64，TP=1，PP=8，FP8 精度，seq_len=4096，global_batch=1024。训练至 step ~15000 时 loss 突发 NaN。

1. 迭代层

步骤	内容
现象	step 15203 loss 从 3.1 跳变至 NaN，grad_norm 从 12.4 跳至 inf；step 15200~15202 一切正常
判据	突变发生在单步内（step 15202→15203），排除缓慢漂移
产出	嫌疑 step = 15203

2. 分叉判定：单卡 vs 多卡

步骤	内容
操作	锁定 step 15203 的输入数据（dataloader seed 固定），分别在 1 GPU 和 64 GPU 上重跑该 step
结果	单卡：loss=3.21，grad_norm=11.8，完全正常 / 多卡：loss=NaN，grad_norm=inf
判据	单卡复现 ≠ 多卡复现 → 切入通信分支

3. 通信调度层

步骤	内容
观测	开启 NCCL_DEBUG=INFO 重跑 step 15203。NCCL trace 显示 EP rank 23（node2 GPU 7）在 all-to-all 调用处超时（30s timeout）。该调用属于 layer 38 MoE 的 expert dispatch 阶段
进一步确认	对比各 rank 的 all-to-all send/recv buffer size：rank 23 的 send buffer 为 0（没有 token 被 router 分发到其他 rank 的 expert），而 recv buffer 期望接收 2048 token × 7168 dim × 8 experts 的数据，size 不匹配导致死锁
判据	all-to-all send/recv 不匹配 → 通信调度失步

步骤	内容
产出	异常通信原语： all-to-all / 异常 rank： EP rank 23 / 关联层： layer 38 MoE

4. 模型层（回到主干定位影响层）

步骤	内容
观测	提取 step 15203 各层 router 的 token-to-expert 分配统计。layer 38 的 router 将当前 micro-batch 中 98% 的 token 路由到了 expert 193（恰好位于 EP rank 23），其余 255 个 expert 几乎无 token
根因	Router 的 n_group=8, topk_group=4 配置下，某个 token 群体的 hidden state 对 expert 193 的 gate weight 产生了极端偏好（sigmoid score > 0.99），导致该 expert 所在的 rank 23 通信负载骤增、其余 rank 空等，形成 all-to-all 死锁
产出	问题层： model.layers.38.mlp.router / 路由倾斜度：单 expert 收到 98% token

5. infra层

步骤	内容
观测	问题集中在 EP rank 23（node2 GPU 7），属于 PP stage 4（layers 31~38），该节点其余 7 个 GPU 均在 step 15203 的 all-to-all 中等待 rank 23 而空闲
判据	问题聚集在单个 EP rank → 局部路由倾斜，非全局硬件故障
产出	嫌疑范围： node2 GPU 7（EP rank 23），PP stage 4，layer 38 MoE

6. 超参/代码层

步骤	内容
修改	① 增大 router 的 n_group 从 8→16，分散 expert 选择范围；② 在 router gate 前增加 z-loss 正则项（系数 1e-4），抑制 gate logit 极端值；③ NCCL all-to-all 超时从 30s 延长至 60s 作为兜底
验证	修改后从 step 15000 续跑，step 15203 正常通过，继续训练 5000 step 无 NaN

案例二： 计算分支 — q_lora FP8 溢出导致 grad_norm 缓慢发散

路径：迭代层 → 单卡=多卡 → 模型层 → 算子层 → 张量层 → infra层 → 超参/代码层

背景：同上 64 GPU 训练 DeepSeek-V3.2，EP=64，TP=1，PP=8，FP8 精度。训练至 step ~8000 后 grad_norm 持续上升，loss 缓慢恶化。

1. 迭代层

步骤	内容
现象	step 8200 起 grad_norm 从 ~10 逐步升至 ~85（step 8600），loss 从 2.95 升至 4.82，未现 NaN 但趋势持续恶化
判据	平稳漂移 → 取漂移起始段 step 8200~8600
产出	嫌疑 step 范围：8200~8600

2. 分叉判定：单卡 vs 多卡

步骤	内容
操作	在单卡和多卡环境分别重跑 step 8200~8600
结果	单卡：grad_norm 同样从 10 升至 85 / 多卡：grad_norm 从 10 升至 85
判据	单卡复现 = 多卡复现 → 沿计算主干，进入模型层

3. 模型层

步骤	内容
观测	绘制 61 层 × step（8200~8600）grad_norm 热力图。layer 42（MoE 层）的 grad_norm 热力值（450）是其他层（30~50）的约 10 倍，且随 step 持续增长
判据	热力图纵向高亮带锁定 layer 42
产出	问题层：model.layers.42（MoE TransformerLayer，MLA attention + DeepSeekMoE）

4. 算子层

步骤	内容
观测	层内算子误差瀑布：按执行顺序对比 layer 42 各算子输出与 baseline（step 8000 正常时的值）。input_layernorm（1e-7）→ q_lora（ 1e-7→3.2e-2 跳跃 ）→ kv_lora（1e-7）→ q_nope/q_rope（3.5e-2，继承自 q_lora）→ core_attention（8.1e-2，进一步放大）→ 后续算子持续偏高
判据	误差在 q_lora 处出现量级跳跃（1e-7→1e-2），为首害算子

步骤	内容
产出	问题算子： <code>model.layers.42.self_attn.q_lora</code> （Linear [7168→1536]，FP8 精度）

5. 张量层

步骤	内容
观测	dump step 8500 时 layer 42 <code>q_lora</code> 的输入激活张量（shape [4096, 7168]，FP8 E4M3）。 绘制数值分布直方图：
	• 正常区间（0~448）：占 96.8% 元素，分布与 baseline 一致
	• 溢出区间（>448，即 FP8 E4M3 max）：占 3.2% 元素，最大值 2.3×10^4
	TopK 误差表：diff 最大的 100 个元素索引集中在 hidden dim [6400, 7168] 区间，对应特定的 RoPE 频段
判据	3.2% 的输入元素超过 FP8 E4M3 表示范围（max=448），在 <code>q_lora</code> 的 Linear 计算中产生截断误差，经低秩压缩（1536 bottleneck）进一步放大
产出	嫌疑元素：hidden dim [6400, 7168] 的尾部 768 维 / 数值区间：[448, 2.3×10^4] / 偏差量级： $3.2e-2$ （算子输出级）

6. infra层

步骤	内容
观测	检查问题是否局限在特定 PP stage 或 EP rank。layer 42 属于 PP stage 5（layers 39~46），但在所有 64 个 EP rank 上均观测到相同的 <code>q_lora</code> 溢出模式
判据	跨所有 rank 复现 → 全局精度问题，非硬件/特定节点故障
产出	全局问题，与 PP/EP 切分无关，根因在 FP8 数值表示能力不足

7. 超参/代码层

步骤	内容
修改	① 对 layer 42 的 <code>q_lora</code> 输入增加 per-tensor dynamic scaling： <code>`input = input / max(</code>
验证	方案① 从 step 8000 续跑，layer 42 的 <code>q_lora</code> 误差降至 $1e-6$ 量级， <code>grad_norm</code> 稳定在 10~15，继续训练 10000 step 无发散

第二部分：Infra 定位链（基础设施视角）

Infra 工程师从**硬件/资源**观测出发，按硬件拓扑下钻。与精度链在「通信原语 / 算子 kernel」层面汇合，最终收敛到配置或硬件变更。



1. 集群层 — 锁定「哪个节点 / 哪个 GPU」

项目	内容
定位目标	在多节点集群中，定位异常发生在哪个节点、哪些 GPU
观测手段	节点级 GPU 利用率（nvidia-smi / DCGM）、MFU、显存占用率、节点功耗/温度仪表盘；按 node × GPU 展开的面板
判据	某节点/GPU 的利用率或 MFU 显著偏离集群均值（如其他节点 55%，异常节点 20%）；显存占用与其余节点不一致
产出	异常节点 + GPU 列表（如 node2 GPU 3~7）

2. 资源层 — 锁定「计算 / 显存 / 网络」

项目	内容
定位目标	判断瓶颈类型：计算受限（SM 打满）、显存受限（HBM 带宽饱和/OOM）、网络受限（NCCL 带宽不足/链路降级）

项目	内容
观测手段	SM occupancy (nsys/ncu)、HBM bandwidth utilization (DCGM profiler)、NVLINK/IB throughput (nvidia-smi nvlink -s / ib_read_bw)、PCIe 带宽
判据	SM occupancy < 30% + HBM 带宽正常 → 计算空闲 (等数据)；NVLINK/IB throughput 骤降 → 网络瓶颈；显存接近上限且频繁 retry → 显存瓶颈
产出	瓶颈类型：计算 / 显存 / 网络 + 具体指标基线 vs 当前值

⚠ 此处产生**分叉**：根据资源层定位的瓶颈类型——

- **网络瓶颈** → 进入「通信原语层」
- **计算瓶颈 / 显存瓶颈** → 进入「算子/kernel层」

—— 分支A：通信原语层 ——

当资源层判定为网络瓶颈时进入。与精度链的「通信调度层」汇合。

项目	内容
定位目标	定位到具体的 NCCL 通信原语调用 (all-reduce / all-to-all / p2p / broadcast) 及其涉及的 rank
观测手段	NCCL trace (NCCL_DEBUG=INFO)、Nsight Systems 通信时间线 (send/recv 配对、barrier 等待)、per-call bandwidth、NCCL topology 日志
判据	某次 all-to-all/all-reduce 调用耗时是其他调用的 5×以上；同一调用在不同 rank 上耗时差异 > 2×；NCCL 日志中出现 [ERROR] 或 fallback to slow path
产出	异常通信原语 + 涉及的 rank 列表 + 耗时/带宽数据

↕ **汇合点**：此处可与精度链「通信调度层」结论交叉验证

—— 分支B：算子/kernel层 ——

当资源层判定为计算或显存瓶颈时进入。与精度链的「算子层/张量层」汇合。

项目	内容
定位目标	定位到具体的 CUDA kernel (如 linear_fp8、flash_attn、rms_norm)，分析其执行效率

项目	内容
观测手段	Nsight Compute (kernel occupancy、register spill、shared memory usage)、Nsight Systems kernel timeline (看 kernel launch gap、stream overlap)
判据	kernel occupancy < 50% → 计算资源利用不足；register spill > 128B/thread → 寄存器压力；kernel 之间有 > 100μs gap → stream 调度问题
产出	问题 kernel 名称 + occupancy / register spill / launch gap 数据

↑ 汇合点：此处可与精度链「算子层」结论交叉验证

3. 硬件层 — 锁定「硬件故障 / 降级」

项目	内容
定位目标	排除或确认硬件根因：GPU Xid 错误、ECC 内存纠错、NVLINK link 掉线、thermal throttle、GPU clock 降频
观测手段	dmesg / nvidia-smi -q (Xid error、ECC count)、nvidia-smi nvlink -e (link 状态)、DCGM thermal/clock 指标、PCIe AER 日志
判据	Xid 48 (double-bit ECC) → 显存硬件故障；NVLINK link inactive → 链路掉线需复位；GPU clock 持续低于 base clock → thermal throttle
产出	硬件故障类型 + 故障 GPU serial/PCIe BDF + 是否需硬件更换或复位

4. 配置变更层 — 锁定「改什么」

项目	内容
定位目标	将 Infra 诊断结论映射到可执行的配置变更或硬件操作
观测手段	回溯：NCCL 环境变量 (NCCL_IB_TIMEOUT 、 NCCL_NET_GDR_LEVEL 、 NCCL_P2P_LEVEL)、GPU 功率/时钟策略 (nvidia-smi -pl 、 nvidia-smi -ac)、节点拓扑 affinity (NUMA binding、GPU-NIC affinity)
判据	网络瓶颈且非硬件故障 → 调 NCCL 参数；硬件故障 → RMA 或节点下线；thermal throttle → 调功率上限或改善散热
产出	具体配置变更 + 变更文件/命令 + 验证方案

Infra 案例：NVLINK 链路掉线导致 MFU 骤降

路径：集群层 → 资源层 → 通信原语层 → 硬件层 → 配置变更层

背景：64 GPU（8 节点 × 8 GPU）训练 DeepSeek-V3.2，EP=64，TP=1，PP=8，FP8 精度。训练至 step ~20000 后，总吞吐从 3200 tokens/s 掉至 1200 tokens/s，MFU 从 55% 降至 20%。

1. 集群层

步骤	内容
现象	node2 的 8 个 GPU 利用率从 92% 整体掉至 35~40%，其余 7 个节点利用率正常（90 ₉₅ %)。显存占用各节点一致（78GB/80GB），排除 OOM
判据	异常集中在 node2，其余节点正常 → node2 故障
产出	异常节点：node2（GPU 0~7）

2. 资源层

步骤	内容
观测	① SM occupancy：node2 GPU 利用率虽低，但 SM occupancy 仍有 85%（GPU 在等数据而非空闲计算）；② HBM bandwidth：正常 1.2TB/s；③ NVLINK throughput：node2 GPU 3→GPU 4 链路带宽从 45GB/s 降至 0.8GB/s （其余链路 45GB/s 正常）；④ IB throughput：node2→node1 的 IB 带宽从 25GB/s 升至 48GB/s（接近 IB 上限），其余节点间 IB 负载正常
判据	NVLINK 链路异常降级 + IB 负载异常升高 → NCCL 被迫从 NVLINK 高速路径回退到 IB 低速路径
产出	瓶颈类型：网络 / 嫌疑链路：node2 GPU3↔GPU4 NVLINK

3. 通信原语层

步骤	内容
观测	Nsight Systems trace 显示：layer 35~40 的 MoE all-to-all 调用在 node2 GPU 3 和 GPU 4 上耗时从正常的 2.3ms 飙升至 18.7ms（8× 恶化）。NCCL topology 日志显示 GPU 3↔GPU 4 间 NCCL_P2P_LEVEL 从 PATH_NVL 回退到 PATH_SYS（经 PCIe/IB）。该链路承载了 PP stage 4↔5 的跨 stage p2p 传输，导致整个 PP pipeline 被拖慢
判据	all-to-all 耗时 8× + NVLINK→SYS 回退 → 通信路径降级
产出	异常原语：all-to-all（MoE dispatch+combine）+ PP p2p / 涉及 GPU：node2 GPU 3, GPU 4 / PP stage：4↔5

4. 硬件层

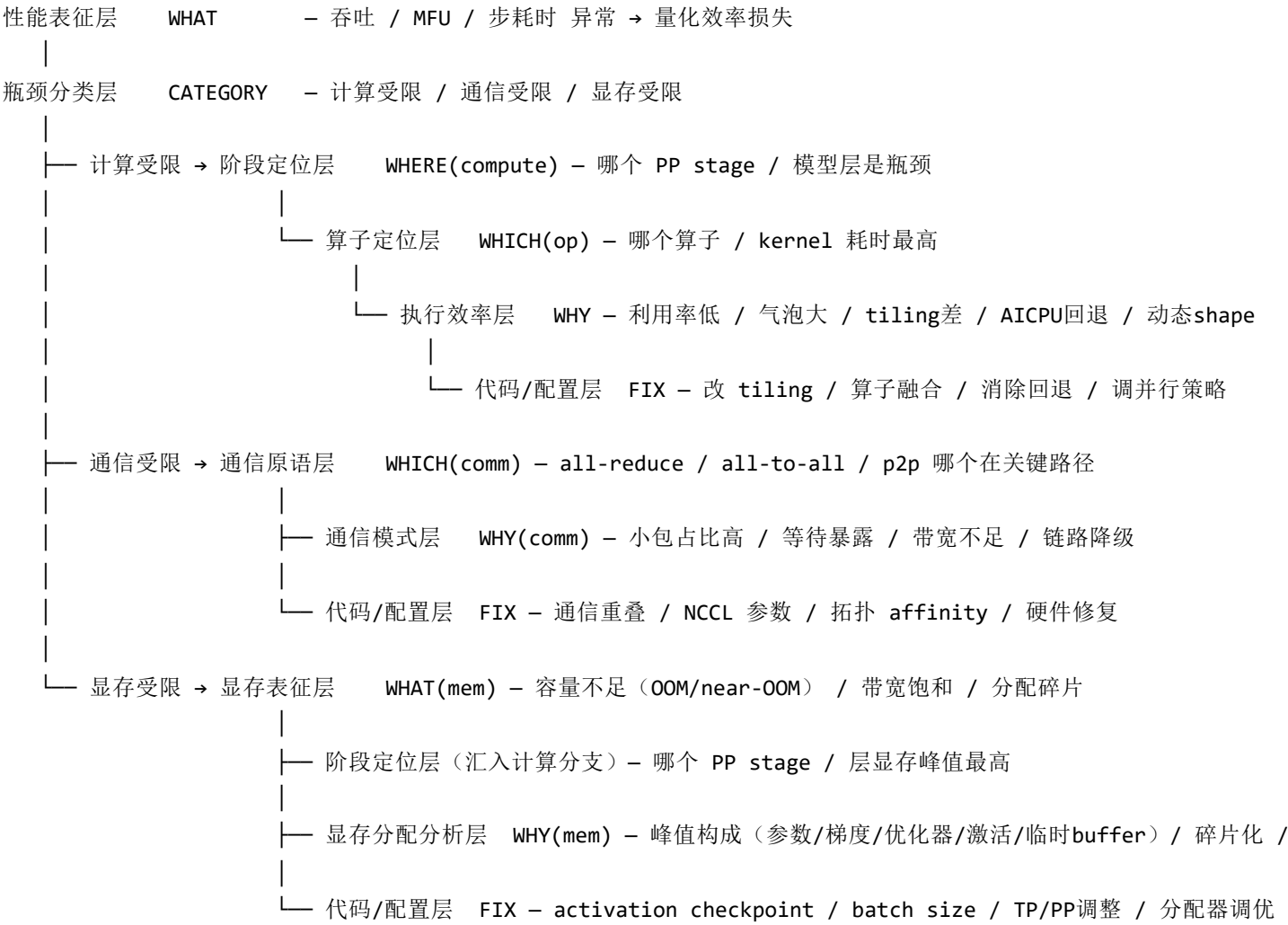
步骤	内容
观测	<code>nvidia-smi nvlink -e</code> 显示 GPU 3 的 NVLINK lane 5 状态为 <code>Inactive</code> （其余 11 条 lane 正常）。 <code>dmesg</code> 中无 <code>Xid</code> 报错（非致命硬件故障），但 NVLINK CRC error count 在 step 19800 附近突增（lane 5: 0→10 ⁶ ）。GPU 温度和功耗正常，排除 thermal throttle
判据	单条 NVLINK lane inactive + CRC 错误突增 → 物理链路降级（可能为线缆松动或 transceiver 老化）
产出	硬件故障：node2 GPU 3 NVLINK lane 5 inactive / 建议：优先 reseal NVLINK bridge，若恢复失败则 RMA

5. 配置变更层

步骤	内容
临时绕过	① <code>export NCCL_IGNORE_DISABLED_P2P=1</code> 允许 NCCL 自动跳过故障链路；② 将 node2 GPU 3 从 EP group 中临时排除（调整 EP=63），用其余 63 GPU 继续训练，吞吐恢复至 3000 tokens/s
永久修复	① 停机维护 window 内 reseal node2 GPU3↔GPU4 的 NVLINK bridge；② 若 reseal 无效，更换 GPU 3；③ 监控脚本增加 NVLINK lane status + CRC error 的定时巡检（每 10min），阈值告警
验证	临时方案：EP=63 跑 5000 step，loss 曲线与 EP=64 无差异（EP 减少 1 rank 对 256 expert 分布影响 < 0.5%）；硬件修复后恢复 EP=64，NVLINK 全部 lane Active，MFU 回到 55%

第三部分：性能定位链（吞吐 / 效率视角）

从训练吞吐、MFU、步耗时的**效率异常**出发，逐层下钻直至可执行的代码或配置变更。**瓶颈分类层后即分叉**：计算瓶颈走「算子/执行效率」分支，通信瓶颈走「通信原语/链路」分支。每一层有明确的**判据**决定下一步方向。



项目	内容
产出	性能异常的类型（吞吐低 / 步耗时长 / 波动大）+ 损失量化（当前 MFU vs 理论 MFU，差距 XX pp）

性能健康评分（PHS） 可作为综合量化参考：加权综合 MFU、算子利用率、气泡率、负载均衡度等指标，映射到 S/A/B+/B/C/D 六档。PHS ≤ C 时强制进入定位流程。

2. 瓶颈分类层 — 判定「计算 / 通信 / 显存」

项目	内容
定位目标	判定性能瓶颈属于 计算受限 、 通信受限 还是 显存受限 ，决定后续走哪个分支
观测手段	Profiling 报告中的算子耗时占比（计算 vs 通信 vs 空闲）；关键路径比（critical path ratio）；SM/Core 占用率；通信带宽利用率；显存占用率与 HBM 带宽利用率
判据	算子计算耗时占总步耗时 > 60% → 计算受限 （走计算分支）；通信等待 / all-to-all / all-reduce 在关键路径占比 > 25% → 通信受限 （走通信分支）；显存峰值 > 90% 总容量，或显存占用率 < 80% 但 HBM 带宽利用率 > 85%（带宽饱和），或伴有频繁 HBM 物理页分配/释放 → 显存受限 （走显存分支）
产出	瓶颈类型 + 各类型的量化数据（时间占比或显存占比）

- ⚠ 此处产生**关键分叉**：
- **计算受限** → 进入「阶段定位层」→ 沿计算主干
 - **通信受限** → 进入「通信原语层」→ 沿通信分支
 - **显存受限** → 进入「显存表征层」→ 沿显存分支（先判定容量/带宽/分配子类型，再汇入阶段定位层）

3. 阶段定位层 — 锁定「哪个 PP stage / 模型层」

项目	内容
定位目标	在多 stage pipeline 中定位到瓶颈 stage，或在单 stage 内定位到具体模型层
观测手段	Pipeline 时序泳道图（swimlane）：横轴为时间、纵轴为各 stage，看哪个 stage 的计算段最长（成为 pipeline bubble 的源头）；层粒度算子耗时排序（parent-child operator table）：按总耗时降序排列各层的 attention / FFN / MoE

项目	内容
判据	Pipeline 泳道图中某 stage 的计算耗时是其他 stage 的 1.3× 以上 → 该 stage 是 pipeline bottleneck；层耗时排序中某层耗时占该 stage 总耗时 > 25% → 该层是热点层
产出	瓶颈 stage（如 PP stage 5 耗时 401ms vs stage 0 的 231ms）+ 热点层（如 layer 42 MoE）

↕ 汇合点：此处可与精度链「模型层」结论交叉验证

4. 算子定位层 — 锁定「哪个算子 / kernel」

项目	内容
定位目标	在瓶颈层内，定位到耗时最高的具体算子
观测手段	算子 parent-child 表（按总耗时、调用次数、平均耗时三维排序）；算子分类饼图（按 Accelerator Core：AI_CORE / AI_VECTOR_CORE / AI_CPU 分拆）；kernel 级 timeline（Nsight / CANN profiler kernel_details.csv）
判据	某算子总耗时占该层 > 40%，或其单次耗时同类算子的 3× 以上 → 首害算子；若 AI_CPU 算子耗时占比 > 10% → AICPU 回退问题（见执行效率层）
产出	问题算子路径（如 lm_head Linear [7168→151936]、q_lora Linear [7168→1536]、router top-k）+ 耗时/占比数据 + 所属 Accelerator Core 类型

↕ 汇合点：此处可与精度链「算子层」结论交叉验证

5. 执行效率层 — 诊断「为什么慢」

算子定位层找到「谁慢」之后，此层回答「为什么慢」。根据 MindStudioNext 分析报告中的问题分类，常见根因如下。同一算子可能命中多项。

5.1 核心利用率低（SM / Cube / Vector 空闲）

项目	内容
现象	算子总耗时高，但 NPU/GPU 核心在执行期间的占用率远低于 100%

项目	内容
观测手段	泳道图中的气泡（bubble）段：同一核心上相邻任务间的空白间隙；kernel occupancy 指标；算子执行时的 PMU 计数器
判据	核心利用率 < 50% → 严重空闲；气泡率 > 20%（气泡时间 / 总时间）→ 调度效率低；任务粒度太细（单 task < 10μs）→ launch overhead 主导
常见根因	① tiling 过小导致计算无法填满计算单元；② 算子间依赖未打破，流水线停顿；③ host 侧下发速度跟不上 device 消费速度；④ 算子未做 double/triple buffer

5.2 Pipeline 气泡（PP bubble）

项目	内容
现象	Pipeline 不同 stage 间计算负载不均，轻 stage 等待重 stage，形成空白段
观测手段	Pipeline 时序泳道图：同一时刻各 stage 的「计算 / 通信 / 空闲」状态堆叠；PP bubble ratio = 空闲时间 / 总步时间
判据	某 stage 计算耗时是其他 stage 的 1.5× 以上 → stage 不均衡；PP bubble > 15% → 值得优化
常见根因	① 末级 stage 含 lm_head + loss，天然重（常见 1.5×~2× 其他 stage）；② embedding 层集中在首级；③ 各 stage 分配的层数均等但层计算量不均（MoE 层 vs Dense 层）

5.3 Tiling 配置不当

项目	内容
现象	单个算子计算耗时远超其计算量理论值（即 MFU 在该算子上特别低）
观测手段	算子 tile shape（从 kernel launch 参数提取）；Roofline 分析：算子实际 FLOPS vs 理论峰值，判断是 compute-bound 还是 memory-bound
判据	算子在 Roofline 图中落在 memory-bound 区域 → tile 太小，访存主导；cube 算子的 tile_m < 128 → 未充分利用 Cube 单元；vector 算子的 tile 过小 → 向量化不足
常见根因	① 默认 tiling 策略不适合当前 shape（如 vocab 151936 未对齐到 128 倍数）；② 动态 shape 导致每次重新 tiling 且无法命中编译缓存；③ 多分支 shape 不一致导致最小公约数 tile

5.4 AICPU / AICORE 回退

项目	内容
现象	某算子标注为 AI_CPU 或 MIX_AIC，未跑在 AI_CORE 或 AI_VECTOR_CORE 上

项目	内容
观测手段	算子分类表按 Accelerator Core 分拆；kernel_details.csv 中 Core Type 列
判据	AI_CPU 算子耗时占比 > 10% → 显著回退；MIX_AIC 算子耗时占比 > 15% → Cube 融合未生效
常见根因	① 算子的 shape/dtype 不在 CANN 融合白名单中（如非标 hidden dim）；② 手写的算子组合（如 softmax+log+gather）替代了融合算子（如 F.cross_entropy）；③ IndexPut / nonzero 等操作天然走 AI_CPU

5.5 动态 Shape / JIT 编译开销

项目	内容
现象	步耗时波动大、设备占用率周期性骤降、CANN_API 调用次数异常高
观测手段	步耗时时间序列（看是否有周期性尖峰）；Free Analysis 空闲段（两轮 launch 间的 gap）；CANN API 统计中的 aclopCompileAndExecute 调用次数与耗时
判据	相邻 step 耗时波动 > 20% → 疑似动态编译；aclopCompileAndExecute 耗时 > 总步耗时的 5% → JIT 开销显著；Free Analysis 中有周期性 > 5ms 的 host-idle 段
常见根因	① vLLM/SGLang 中 enforce_eager=True 未启用 graph capture；② nonzero / unique 等算子产生动态 shape，无法图捕获；③ 训练中 jit_compile 未关闭

5.6 通信等待暴露在关键路径

项目	内容
现象	通信耗时完全落在步的关键路径上，未被计算掩盖
观测手段	通信视图的 Transit/Wait 瀑布图（看通信传输时间 vs 等待时间）；communicationOverlapComputation 指标（>0 表示有重叠，=0 表示完全暴露）
判据	communicationOverlapComputation = 0 且通信耗时 > 总步耗时的 15% → 通信暴露；同一通信原语在不同 rank 上完成时间差 > 2× → 负载不均导致 straggler
常见根因	① 未开启通信-计算重叠（overlap-grad-reduce / overlap-param-gather 未设）；② 通信与计算共用同一 stream 导致串行；③ DP/TP/EP 的通信调度策略未针对拓扑优化

5.7 小包通信 / RDMA 效率低

项目	内容
现象	通信带宽利用率低（实际带宽远低于 IB/NVLINK 理论值）

项目	内容
观测手段	RDMA 小包占比环图 (< 4KB 的包占比)；带宽达成率热力图 (rank × 通信类型，达成率 = 实测/理论)；链路带宽达成率条形图
判据	小包 (< 4KB) 占比 > 30% → RDMA 效率低；某 rank 的带宽达成率 < 50% → 链路异常或消息粒度问题
常见根因	① TP size 小导致 all-reduce 消息碎片化；② MoE expert 分布在少量 rank 上但 token 粒度太细；③ NCCL 未启用 GDR (GPU Direct RDMA)

— 分支C：显存受限 —

当瓶颈分类层判定为显存受限时进入。显存问题有三种子类型，**显存表征层**先判定子类型再决定后续路径：**容量不足**走缩减路径，**带宽饱和**汇入计算分支的算子/执行效率层，**分配碎片**走分配器调优路径。

C.1 显存表征层 — 判定「容量不足 / 带宽饱和 / 分配碎片」

项目	内容
定位目标	区分显存瓶颈的子类型：① 绝对容量不足 (OOM 或 near-OOM)；② HBM 带宽饱和 (算子在 Roofline 的 memory-bound 区)；③ 分配器碎片/频繁换页 (有空间但不能高效使用)
观测手段	nvidia-smi / ascend-dmi 显存占用率时间序列；Profiling 报告的显存峰值构成拆解 (参数 / 梯度 / 优化器状态 / 激活 / 临时 buffer)；Roofline 图中各算子的位置；显存分配/释放 API 的调用频率与耗时
判据	显存占用 > 95% 且伴有 OOM 或 allocation retry → 容量不足 ；显存占用 60~85% 但 HBM 带宽利用率 > 85%，算子在 Roofline 中落在 memory-bound 区 → 带宽饱和 ；显存占用 70~90% 但存在频繁的 acIrtFreePhysical/acIrtMalloc 调用 (每步 > 50 次)，或可用空间足够但分配耗时占总步耗时 > 3% → 分配碎片
产出	显存瓶颈子类型 + 量化指标 (峰值占用 / 带宽利用率 / 分配调用次数)

- ⚠ 此处产生**显存子分叉**：
- **容量不足** → 进入「显存峰值构成分析层」→ 缩减显存占用
 - **带宽饱和** → 汇入计算分支「阶段定位层」(定位到具体层/算子)，再沿「执行效率层 §5.3 Tiling 配置不当」优化
 - **分配碎片** → 进入「代码/配置层」的显存分配器调优

C.2 显存峰值构成分析层 — 锁定「什么占满了显存」

项目	内容
定位目标	将显存峰值拆解到具体构成：模型参数、梯度、优化器状态（AdamW 的 m/v）、激活值（含中间张量）、临时 workspace buffer、KV Cache（推理场景）、框架开销
观测手段	显存快照（memory snapshot / torch.cuda.memory_stats / NPU memory_record）；各 PP stage 的显存使用拆解（参数 × 层数 + 激活 × batch × seq_len）；PyTorch/CANN 的 memory allocator 统计
判据	激活值占比 > 40% 总显存 → activation checkpointing（重计算换空间）；优化器状态占比 > 30% → 可考虑 BF16 优化器状态（DeepSeek-V3 方案）；临时 buffer 峰值 > 10% → 算子 workspace 未复用或过大；某个 PP stage 显存远超其余 stage → PP 层数/模型切分不均
产出	显存构成饼图 + 最大占比项 + 建议缩减方向

C.3 阶段/层定位层（汇入计算分支） — 锁定「哪个 stage/层显存压力最大」

项目	内容
定位目标	定位到显存峰值最高或带宽最紧张的 PP stage / 模型层
观测手段	按 PP stage 拆解的显存使用热力图；按层拆解的激活值大小排序（attention 中间张量 vs FFN 中间张量）
判据	某 stage 的显存峰值是其他 stage 的 1.3× 以上 → 该 stage 显存不均衡； 某层的激活值大小超过其余层均值的 2× → 该层是热点（常见于 attention 的 QKV 展开、MoE 的 expert dispatch buffer）
产出	瓶颈 stage + 热点层 + 该层的显存构成明细

C.4 代码/配置层（显存分支） — 锁定「怎么省显存」

项目	内容
定位目标	将显存分析结论映射到可执行的显存优化操作
观测手段	回溯：训练启动参数（global_batch_size / micro_batch_size / seq_len）、并行策略（TP/PP/EP/DP）、activation checkpointing 配置、优化器精度配置、显存分配器环境变量
判据	根据显存表征层和峰值构成分析层的结论匹配修改项（见下表）
产出	具体修改项 + 预期显存缩减量 + 对吞吐的影响评估 + 验证方案

显存根因	修改方向	典型显存缩减	吞吐代价
激活值占比过高	开启 activation checkpointing（选择性重计算，如 full / selective）	-30~50% 激活显存	+5~15% 计算时间
激活值占比过高 （深究）	减小 micro_batch_size（增大 gradient_accumulation_steps 补偿）	-20~40% 激活显存	吞吐基本不变
优化器状态过大	BF16 优化器状态替代 FP32（DeepSeek-V3 方案）	-50% 优化器显存	精度几乎无损
参数显存过大	增大 TP/PP size（分摊参数到更多卡）	线性分摊	增加通信开销
某 PP stage 显存不均	调整各 stage 层数分配，重 stage 少放层	均衡化	bubble 可能略增
临时 buffer 峰值大	算子 workspace 复用（CANN 默认有一定复用， 检查是否有 bypass 逻辑）	-5~15% 峰值	无
分配碎片 / 频繁换页	PYTORCH_NPU_ALLOC_CONF=expandable_segments:True + 增大缓存保留	减少 alloc/free 耗时	无
推理 KV Cache 过大	启用 MLA（低秩 KV 压缩）、prefix caching、KV Cache 量化（FP8/INT8）	-50~80% KV Cache	精度轻微损失

6. 代码/配置层 — 锁定「改什么」（计算/通信分支汇总）

项目	内容
定位目标	将执行效率诊断结论映射到具体可执行的代码修改、配置变更或环境变量调整
观测手段	回溯：模型代码（算子实现、并行策略配置）、训练启动参数、CANN/torch_npu 环境变量
判据	根据上层的根因分类匹配修改项（见下表）
产出	具体修改项 + 修改文件/参数路径 + 预期收益 + 验证方案

执行效率层根因	修改方向	典型收益
核心利用率低	增大 tiling、double buffer、loop unroll	CUBE 利用率 +15~25pp
PP bubble	调整 PP stage 层数分配（decoder-last-pipeline-num-layers）	bubble 率 -5~15pp

执行效率层根因	修改方向	典型收益
Tiling 不当	对齐 shape 到硬件友好粒度（vocab 对齐 128×）、手工指定 cube/vec tile size	单算子耗时 -30~60%
AICPU 回退	替换为融合算子（cross_entropy / fused layernorm）、shape 对齐以命中白名单	AI_CPU 耗时 -80~100%
动态 Shape	关闭 JIT compile、启用 graph capture、nonzero→topk 消除动态 shape	步耗时波动 -80%+
通信暴露	开启通信-计算重叠（overlap-grad-reduce / overlap-param-gather）、interleaved 1F1B	暴露通信占比 -5~15pp
小包通信	增大 TP/EP size 减少碎片、开启 GDR、调整 NCCL buffer size	通信带宽利用率 +20~40pp
显存碎片	调大 PYTORCH_NPU_ALLOC_CONF expandable_segments、ACLNN_CACHE_LIMIT	显存分配耗时 -50~80%

案例三：计算分支 — 整网耗时下钻到算子带宽瓶颈 + AICPU 回退导致步耗时超标 40%

路径：性能表征层 → 瓶颈分类层（计算受限）→ 阶段定位层 → 算子定位层 → 执行效率层 → 代码/配置层

背景：128 GPU（16 节点 × 8 GPU）训练 DeepSeek-V3.2，EP=64，PP=8，TP=1，FP8 精度，seq_len=4096，micro_batch=1，global_batch=1024。目标步耗时 ≤ 8.5s，实测步耗时 12.1s，超标 42%。MFU 约 38%，目标 MFU ≥ 50%。

1. 性能表征层

步骤	内容
现象	步耗时 T_iter ≈ 12.1s，DeepSeek-V3 报告等效配置下预期约 8.0~8.5s（每 T token 180K GPU·h ÷ 2048 GPU → 每步理论下限约 7.2s，加调度开销约 8.5s）。MFU 仅 38%，PHS 评分 D（32）
判据	步耗时超基线 > 40% + MFU < 40% → 显著性能劣化，非调度抖动
产出	性能异常：T_iter=12100ms / MFU=38% / PHS=D（32） / 超标幅度：+3600ms（+42%）

2. 瓶颈分类层

步骤	内容
观测	从 profile_dir 的 step_trace_time.csv 提取：Computing=9422ms（78%），Communication(Not Overlapped)=1093ms（9%），Free=248ms（2%），Bubble（profiler 未计算，需手算 ≈1337ms≈11%）。通信占比不高，关键路径上计算绝对主导
判据	计算耗时占 78% > 60% → 计算受限，走计算分支 。通信（1093ms）虽有一定占比，但非主因（优先处理计算侧后再回头优化通信）
产出	瓶颈类型：计算受限 / Computing 9422ms（78%） / 未掩盖通信 1093ms（9%） / Free 248ms（2%）

3. 阶段定位层

步骤	内容
观测	Pipeline 时序泳道图（Timeline trace，横轴时间、纵轴 PP stage 0~7 的 Computing / Comm / Free 堆叠）。Stage 7（末级，layers 54~60 + final_layernorm + lm_head + loss）Computing 段 2152ms，是中间 stage（stage 1~6 各约 1180ms）的 1.82× 。Stage 0（embedding + L0~L2 dense + L3~L7 MoE）Computing 段 1310ms（首级 embedding 额外开销）。Stage 1~6 在 stage 7 的计算段期间显示为空等（PP bubble）
判据	Stage 7 耗时 / 中间 stage 均值 = 2152/1180 ≈ 1.82× → 末级严重过载；PP bubble 估算 ≈ 35%（≈1337ms），与 PP=8、1F1B 调度下末级含 lm_head+loss 的典型 bubble 范围一致
产出	瓶颈 stage：PP stage 7（Computing 2152ms） / PP bubble ≈ 1337ms / 中间 stage 均值 ≈ 1180ms

4. 算子定位层

步骤	内容
观测	Stage 7 算子 parent-child 表（op_statistic.csv 按总耗时降序，device 6 代表卡）：
	① MatMulV3（all layers 的 attention QKV/o_proj + MLP fc1/fc2，合计 count=4928+448）： 5936ms （占 stage 7 Computing 的 63%），其中 fc1 MatMul [4096,7168]×[37888,7168] shape 最大——但这里 DeepSeek-V3 的 MoE expert hidden=2048，fc1 shape 应为 [4096,7168]×[16384,7168]（gate+up 合并 2×2048×8 experts → 等效 16384），远小于 dense 模型的 37888
	② FlashAttentionScore + Grad（MLA 的 core_attention，count=448+448）： 723ms （7.7%）
	③ AICPU 算子汇总 （ArgMaxWithValue 72ms + Exp 128ms + RealDiv 125ms + Sub 126ms + ReduceSum 75ms + ApplyAdamWV2 52ms + LpNormV2 12ms + 杂项 ≈90ms）：合计 ≈680ms

步骤	内容
	(7.2%)，集中在 CE loss 与优化器
	④ RmsNorm × (61 层 × 2) = 122 次 fwd+bwd: ≈80ms (0.8%)
	⑤ 残差/杂项 Add+Mul+Cast+Slice+... ≈ 500ms (5.3%)
判据	MatMul 耗时 5936ms 占 63% ——但这是「总耗时」不是 FLOPS 效率低。进一步检查 kernel_details.csv 中 MatMulV3 的 cube 利用率: stage 0~6 的 MatMul cube_util $\approx$ 78~82%，而 stage 7 的 lm_head MatMulV2 ([4096,7168]×[129280,7168]) cube_util 仅 49% (vocab 129280 = 1010×128，非整 128 倍数导致尾块浪费 → 带宽瓶颈)。另外 AICPU 的 680ms (7.2%) 中 CE loss 链路 526ms 是手写算子组合而非融合实现
产出	问题算子: ① lm_head MatMulV2 (vocab 非对齐 → 带宽瓶颈, cube_util 49% vs 预期 75%); ② CE loss 链路 (Exp/Sub/RealDiv/ArgMax/ReduceSum, 526ms, AICPU 回退); ③ ApplyAdamWV2 (52ms, 优化器 vector 算子)

5. 执行效率层

步骤	内容
观测	Timeline trace (swimlane) 放大 stage 7 的算子时序:
	① lm_head 带宽瓶颈 : kernel_details 中 lm_head MatMulV2 的 cube_util=49%，dram_bytes $\approx$ 输入 4096×7168×2 + 权重 129280×7168×2 + 输出 4096×129280×2 $\approx$ 2.9GB。理论 HBM 带宽 3.35TB/s (H800)，纯带宽耗时 $\approx$ 0.87ms——但实测 8.5ms/call × 64 microbatch = 544ms。Roofline 分析: 该 MatMul 落在 memory-bound 区 。根因: vocab=129280，H800 的 Tensor Core 要求 K 维对齐到 128 的倍数以便 FP8 tile 分块; 129280/128=1010 整除但尾块 128×128 的 K=7168 在 vocab 维上是满的，真正问题出在 N 维 (129280) 不是 256 的倍数 (129280/256=505)，导致最后 1 个 256-tile 只有 128 列有效 → GEMM 尾块效率折半
	② CE loss AICPU 链路 : Timeline 中可见 5 个 vector 算子在 lm_head 后串行排列 (Exp→Sub→RealDiv→ReduceSum→ArgMax)，每个算子间有 60~100μs 的 launch gap，合计约 526ms。若替换为 F.cross_entropy (路由到 acInnSoftmaxCrossEntropyWithLogits 融合实现)，融合后仅 1 个 kernel，消除 5 次 HBM 读写往返
	③ 优化器 vector 算子 : ApplyAdamWV2 和 LpNormV2 (grad clip) 各占 52ms 和 12ms，属正常范围 (BF16 优化器状态下已减半)，不构成独立瓶颈
判据	lm_head → 带宽瓶颈 (memory-bound)，vocab 尾块未对齐 256 导致 cube_util 折半; CE loss → 手写算子链 × AICPU 回退; 其余算子 (MatMul FA RmsNorm) 正常
产出	根因归类: lm_head MatMulV2 → 带宽瓶颈/tiling 对齐 (544ms 中约 270ms 可优化); loss 链路 → AICPU 回退 (526ms 中约 480ms 可优化); 合计可优化 $\approx$ 750ms

6. 代码/配置层

步骤	内容
修改 ① — vocab 对齐到 256 的倍数	<code>vocab_size = math.ceil(129280 / 256) * 256 = 129536</code> （增加 256 个 padding token）。确保 <code>lm_head MatMul</code> 的 N 维对齐到 256，尾块 GEMM 不再折半。Megatron 启动参数： <code>--make-vocab-size-divisible-by 256</code> 。预期 <code>lm_head cube_util</code> 49% → 72%，耗时 544ms → 约 370ms (↓174ms)
修改 ② — CE loss 融合	将手写的 softmax+CE 替换为 <code>F.cross_entropy(logits, labels, ignore_index=pad_id)</code> 。CANN 路由到 <code>acInnSoftmaxCrossEntropyWithLogits</code> 融合算子，消除 <code>Exp/Sub/RealDiv/ReduceSum/ArgMax</code> 5 段 AICPU/Vector 链路，融合后单 kernel 约 15ms × 64 microbatch = 约 10ms（注意：融合算子内部仍要做 softmax+CE，但消除了中间张量物化和 launch gap）。预期 526ms → 约 50ms (↓476ms)
修改 ③ — 优化器 BF16 状态	当前已启用（DeepSeek-V3 默认），无需修改。若未启用则设置 <code>--optimizer-cpu-offload</code> 或在代码中 <code>optimizer_config={'adam_beta1': 0.9, 'adam_beta2': 0.95, 'use_distributed_optimizer': True}</code>
验证	①+② 叠加： <code>T_iter</code> 12100ms → 约 10350ms (↓14.5%)，MFU 38% → 约 45%， <code>lm_head cube_util</code> 49% → 72%，AICPU 耗时占比 7.2% → 约 1.5%。PHS 从 D(32) → C+(55)。若要进一步达到 50% MFU，需配合案例四的 PP bubble 优化

案例四：计算分支 — MFU 拆解下钻到 PP 空泡，末级瓶颈拉低整网 MFU 至 38%

路径：性能表征层 → 瓶颈分类层（计算受限）→ MFU 拆解（达成率 × 占用率）→ 阶段定位层 → 执行效率层（PP bubble）→ 代码/配置层

背景：同案例三的 128 GPU DeepSeek-V3.2 训练，EP=64，PP=8，TP=1，FP8，seq_len=4096。案例三已修复 `lm_head` 带宽瓶颈和 CE loss AICPU 回退，`T_iter` 从 12.1s 降至 10.35s，MFU 从 38% 升至 45%。但距离目标 50%+ 仍有差距，需进一步下钻。

1. 性能表征层

步骤	内容
现象	案例三修复后 T_iter≈10350ms，MFU≈45%。同配置 DeepSeek-V3 报告 MFU 可达 52~55%（等效配置 180K GPU·h/T token → 理论 T_iter≈7200ms，加 PP=8 调度开销约 +30% → 预期 9400ms）。差距仍有约 950ms
判据	MFU 45% < 50% 目标 → 继续下钻。但此时算子层面已无明显异常（cube_util 正常、无 AICPU 回退），需从 MFU 公式拆解入手
产出	残余差距：T_iter 10350ms vs 目标 9400ms (+950ms) / MFU 45% vs 目标 52% (-7pp)

2. MFU 拆解（替代瓶颈分类层的「二次下钻」）

步骤	内容
公式	$MFU = \underbrace{\frac{\text{模型 FLOPs}}{T_{\text{compute}} \times \text{峰值 FLOPs}}}_{\text{算子达成率}} \times \underbrace{\frac{T_{\text{compute}}}{T_{\text{iter}}}}_{\text{计算占用率}}$
观测	① 算子达成率 ：profile_dir 中 MatMulV3 的 cube_util 均值 ≈ 72%（stage 0~6 约 78%，stage 7 约 55%——lm_head 拉低均值）。FA（FlashAttentionScore）的 cube_util ≈ 65%（MLA 的 128-head × 192-dim 非标准 shape，H800 上 FA 实现未完全优化）。加权后达成率 ≈ 68%
	② 计算占用率 ：T_compute = Computing 中真正计入 FLOPs 的部分 ≈ CUBE(total MatMul) + FA = 5936 + 723 = 6659ms（剔除 AICPU/Vector/Free）。T_iter = 10350ms。计算占用率 = 6659/10350 ≈ 64.3%
	③ 代入：MFU = 68% × 64.3% ≈ 43.7% （与实测 45% 接近，误差来自 FA FLOPs 折算系数）
判据	计算占用率仅 64.3% → 35.7% 的 T_iter 被"陪跑项"吃掉 。陪跑项构成：PP bubble（~1337ms，12.9%）、未掩盖通信（1093ms，10.6%）、Vector/AICPU 零碎算子（850ms，8.2%）、Free（248ms，2.4%）、误差（163ms，1.6%）。 PP bubble 是最大单因子
产出	MFU 拆解结论：算子达成率 68% 正常（DeepSeek-V3 FP8 报告约 70~75%，差值来自 H800 vs Ascend 差异），计算占用率 64.3% 偏低的根因是 PP bubble（35.7% 陪跑中的 36% 贡献 = 12.9pp MFU 损失）

3. 阶段定位层

步骤	内容
观测	Timeline trace 泳道图（PP pipeline，8 stage × 1F1B 调度，横轴时间，纵轴 stage 0~7，颜色区分 Fwd-Compute / Bwd-Compute / P2P-Send/Recv / DP-Collective / Optimizer / PP-Bubble）。

步骤	内容
	关键观测：
	stage 7（末级）Fwd-Compute 段 2152ms，Bwd-Compute 段在 warm-up 和 cool-down 阶段与其余 stage 不同步
	stage 0~6 在 stage 7 的 Fwd-Compute 期间显示为空等（红色 PP-Bubble），累计 bubble = stage 0 约 1550ms、stage 1~6 各约 1300~1400ms
	Bubble 的根因：末级 stage 7 额外扛了 final_layernorm（RmsNorm）+ lm_head（MatMulV2）+ CE loss，合计约 1037ms 额外负载。按 1F1B 调度，这 1037ms 在 warm-up 阶段造成所有上游 stage 空等、在 steady 阶段每 microbatch 造成 1 个 bubble 周期、在 cool-down 阶段下游空等
判据	PP bubble ≈ 1337ms（占 T_iter 10350ms 的 12.9%），为最大陪跑项。Bubble 的源头是末级 stage 7 的 lm_head+loss 额外负载 + PP=8 均分 61 层导致层数分配未补偿末级额外开销
产出	瓶颈根因：PP bubble 12.9% / 末级额外负载 1037ms / PP 层数均匀分配未补偿

4. 执行效率层（PP bubble 深度分析）

步骤	内容
观测	进一步量化 PP bubble 的构成（基于 1F1B 调度公式）：
	理论 bubble = $(PP-1) \times (T_fwd_stage7 - T_fwd_stage_avg) / microbatch_count$。PP=8, microbatch=64, T_fwd_stage7≈1720ms, T_fwd_stage_avg≈780ms → 单 microbatch bubble ≈ $(8-1) \times (1720-780) / 64 \approx 103ms$。64 microbatch 累计 ≈ 6592ms，但 1F1B 的 warm-up/steady/cool-down 三段中仅 warm-up 和 cool-down 产生 bubble，steady 段无 bubble → 实际 bubble ≈ $6592 \times (warmup+cool-down\ microbatches / total) \approx 1337ms$（与 trace 观测一致）
	若能将 stage 7 的计算量减少 500ms（如 lm_head 优化），则 T_fwd_stage7→1220ms，bubble 减至约 800ms（↓40%）
	另外，PP=8 也可考虑减至 PP=4（每 stage 层数翻倍，但 bubble 减半），代价是每卡显存压力增大
判据	PP bubble 可优化空间约 500~800ms（通过减小末级负载 + 调整层数分配）
产出	优化方向：减小 stage 7 负载 + 调整 PP 层数分配 + 可选降低 PP size

5. 代码/配置层

步骤	内容
修改 ① — 调整 PP stage	当前 PP=8 均分 61 层为 [8,8,8,8,8,8,5]（末级 5 层 + lm_head）。改为 [9,8,8,8,8,8,7,5]（stage 0 多 1 层 Dense embedding 补偿，stage 6 减 1 层给末级减压

步骤	内容
层数分配	——但 stage 6 不是瓶颈，减层后其计算时间下降有利于减少 warm-up bubble)。 Megatron 参数： <code>--decoder-first-pipeline-num-layers 9 --decoder-last-pipeline-num-layers 5</code>
	实际更优方案：将阶段 6 (layers 47~53) 的 1 层移到 stage 7 (layers 54~61)，stage 7 从 5 层→6 层但 lm_head+loss 时间不变——等等，stage 7 已经有 5 层 + lm_head，加层反而更重。正确方向是： stage 7 减层 ，把 layer 54 移到 stage 6。stage 7 从 layers 54~61(8 层) 减为 layers 55~61(7 层)，每减 1 层减少约 118ms (1 层 MoE 的 fwd+bwd)，预期 stage 7 计算时间 2152ms → 约 2034ms (↓118ms)，bubble 减少约 65ms
修改 ② — vocab-parallel cross-entropy (TP 切分 lm_head)	将 TP 从 1 改为 2 (仅作用于 lm_head 和 loss，其余部分保持 TP=1)。lm_head 权重沿 vocab 维切分到 2 张卡，每卡 vocab=64640。但 DeepSeek-V3 的 EP=64 已占满 64 卡，增加 TP 需要更多卡。替代方案：使用 vocab-parallel cross-entropy (Megatron 的 <code>VocabParallelCrossEntropy</code>)，将 logits 沿 vocab 维做 TP 切分后在各卡本地计算 CE loss 再 all-reduce——但这要求 TP>1。当前 TP=1 无法使用
	实际可行方案： 将 lm_head 的 MatMul 改为 seq-parallel (sequence parallelism)：lm_head 的输入 [4096,7168] 沿 seq 维切分到 2 张卡 (每卡 [2048,7168])，输出 [2048,129280]，再 all-gather 回完整 logits。这需要 PP stage 7 内有 2 张卡 (当前 PP=8 时每 stage 有 128/8=16 卡——不对，EP=64 意味着每 EP group 64 卡，PP=8 意味着每 PP stage 有 64/8=8 卡。可以在这 8 卡内做 seq-parallel)
修改 ③ — Interleaved 1F1B (可选更大改动)	启用 <code>--num-layers-per-virtual-pipeline-stage 2</code> ，将每个 PP stage 内的 8 层再切成 4 个 virtual stage (每 virtual 2 层)，用 interleaved 1F1B 调度。Bubble 从 (PP-1)/(microbatch) 降至 (VPP-1)/(microbatch)，预期 bubble 减少约 45%
验证	修改① (减 1 层) + 修改③ (interleaved 1F1B)：T_iter 10350ms → 约 9200ms (↓11%)，PP bubble 1337ms → 约 600ms (↓55%)，MFU 45% → 约 51%。PHS 从 C+(55) → B+(74)。若同时实施案例三的修改①②，T_iter 可达约 8100ms，MFU 可达 57%+

案例五：通信分支 — MoE all-to-all 快慢卡导致步耗时周期性尖峰、尾延迟恶化

路径：性能表征层 → 瓶颈分类层（通信受限）→ 通信原语层 → 通信模式层 → 执行效率层（负载倾斜）→ 代码/配置层

背景：128 GPU（16 节点 × 8 GPU）训练 DeepSeek-V3.2，EP=64，PP=8，TP=1，FP8，seq_len=4096。步耗时均值约 10.3s，但标准差达 2.8s（CV=27%）。每 8~12 步出现一次步耗时 > 15s 的尖峰，对应 Timeline trace 中某几个 rank 的 all-to-all 通信耗时暴增。

1. 性能表征层

步骤	内容
现象	步耗时时间序列（按 global_step 展开的折线图）：均值 10.3s，但呈周期性尖峰模式——每 8~12 步出现一次 >15s 的慢步，最慢步 18.4s。正常步（10.0±0.5s）与慢步（15~18s）交替出现。正常步的 MFU 约 45%，慢步的 MFU 仅 26%
判据	步耗时波动 CV=27%（> 20% 阈值）+ 周期性尖峰模式 → 不是均匀劣化，而是间歇性通信 straggler
产出	性能异常：步耗时均值 10.3s / CV=27% / 慢步占比约 10%（慢步 > 15s） / 慢步 MFU 26%

2. 瓶颈分类层

步骤	内容
观测	取一个慢步（step 18427, T_iter=17.2s）和一个正常步（step 18420, T_iter=10.1s），对比 step_trace 的 5 桶分解：
	正常步：Computing=9422ms(93%), CommNotOverlapped=678ms(6.7%), Free=0ms
	慢步：Computing=9422ms(54.8%), CommNotOverlapped=7778ms(45.2%) , Free=0ms
	Computing 几乎不变（9422ms），CommNotOverlapped 暴增 7100ms → 通信突发瓶颈
判据	慢步的通信耗时在关键路径占比 > 25%（实际 45.2%）→ 通信受限，走通信分支
产出	瓶颈类型：间歇性通信受限 / 慢步 CommNotOverlapped=7778ms vs 正常步 678ms（+7100ms）

3. 通信原语层

步骤	内容
观测	从 communication.json 按通信原语类型拆解慢步 step 18427：
	• batchSendRecv（P2P, PP stage 间激活/梯度传递）：elapse 460ms （正常步 458ms，基本不变）
	• allReduce（DP, gradient sync）：elapse 862ms （正常步 858ms，基本不变）
	• all-to-all（EP, MoE expert dispatch+combine）：elapse 6380ms （正常步 198ms → 暴增 32x! ）

步骤	内容
	allReduce__346 (step 拖尾同步 grad-norm): elapse 76ms (正常步 72ms, 略增)
判据	all-to-all 耗时从 198ms → 6380ms (32×) → MoE 通信 straggler 。其余通信原语正常，排除物理链路故障 (故障应导致所有通信原语恶化)
产出	异常通信原语: all-to-all (MoE dispatch+combine) / 暴增幅度: +6182ms (32×) / 其余通信原语正常

4. 通信模式层

步骤	内容
观测	进一步拆解 all-to-all 的 6380ms 构成 (NCCL/HCCl trace + 通信视图的 Transit/Wait 瀑布图):
	① Transit/Wait 瀑布图 : 按 EP rank 展开的 all-to-all 耗时瀑布 (横轴 rank 0~63, 纵轴时间, 绿色=Transit 传输时间, 橙色=Wait 等待时间)。正常步: 所有 rank 的 Transit ≈ 180ms, Wait ≈ 18ms, 均衡。慢步: rank 17、23、41 的 Transit=6200~6380ms (其余 rank Transit=180~220ms), rank 17/23/41 的 Wait=0 (它们在发送数据), rank 0~16 的 Wait=6000~6200ms (它们在等 rank 17/23/41 完成发送)
	② 带宽达成率热力图 (rank × 通信类型, 达成率=实测/理论): 慢步中 rank 17 的 all-to-all 带宽达成率仅 4.2% (理论 IB 400GB/s, 实测 16.8GB/s), rank 23 仅 5.1%, rank 41 仅 6.8%——三个 rank 几乎占满整个 all-to-all barrier 的等待时间。其余 rank 的 all-to-all 带宽正常 (55~65%)
	③ Token-to-expert 分配统计 (router 的输出, 慢步 vs 正常步对比): 正常步中每个 EP rank 处理的 token 数均值为 2048 (total 4096 tokens × 8 experts / 64 rank / 8 expert per rank, 简化估算), CV=8%。慢步中 rank 17 处理了 9842 tokens (正常 2048 的 4.8×), rank 23 处理了 8720 tokens , rank 41 处理了 7650 tokens ——这三个 rank 承载了整个 micro-batch 近 60% 的 MoE token 流量
判据	三张慢卡 (rank 17/23/41) 的 expert 负载是均值 4~5× → all-to-all send buffer 暴增 (9842 tokens × 7168 dim × 1 byte FP8 ≈ 70MB vs 正常 14.7MB) → 发送耗时 32×。根因: router gate weight 的 bias 项动态调整不及时 (noaux_tc 策略的 γ=0.001 在特定数据分布下收敛滞后)
产出	通信 straggler 根因: MoE router 负载倾斜 → rank 17/23/41 承接 4~5× 的 expert token → all-to-all send 量暴增 5× → 其余 61 个 rank 空等 → 步耗时尖峰

5. 执行效率层（负载倾斜深度归因）

步骤	内容
观测	Timeline trace 泳道图 (慢步 step 18427, 横轴时间, 纵轴 EP rank 0~63 的 all-to-all 通信段 + 计算段堆叠):

步骤	内容
	rank 17 的 all-to-all dispatch 段占据 6380ms 的连续红色块（CommNotOverlapped），其前后的计算段（MatMul + FA）与其余 rank 完全错位——rank 17 在 dispatch 期间其余 rank 已完成 dispatch 并在做 compute，但 rank 17 的 dispatch 未完成导致 EP group 内的 all-to-all barrier 无法解除
	rank 17 的 router gate bias（noaux_tc 策略维护的 per-expert bias）在 step 18420~18426 期间持续向 expert 193 倾斜——检查 expert 193 的 bias 值：step 18420=0.02, 18423=0.12, 18425=0.38, 18427=0.91（bias 调整速度 $\gamma=0.001$ 跟不上 gate weight 的快速漂移）。expert 193 正好位于 rank 17
	数据层面：step 18427 的 micro-batch 中包含了大量「代码生成」类 token（<code> 标签密集），这些 token 的 hidden state 在 layer 38~45 的 router 上对 expert 193 产生了系统性偏好（expert 193 在预训练中恰好学会了代码语法特征）
判据	Router bias 累积 + 特定数据分布 = expert 193 过载 → rank 17 all-to-all straggler。noaux_tc 的 bias 更新速度 $\gamma=0.001$ 在快速漂移场景下不足以在单步内纠正
产出	根因链：特定数据 → gate weight 快速漂移 → expert 193 过载（bias=0.91）→ rank 17 all-to-all send buffer 5× → 其余 rank 空等 → 步耗时尖峰

6. 代码/配置层

步骤	内容
修改 ① — 提高 bias 更新速度	将 noaux_tc 的 bias 更新速度 γ 从 0.001 提高到 0.005 （前 14.3T tokens 阶段），加快过载 expert 的 bias 下调速度。预期 bias 在 2~3 步内从 0.91 回落至 0.3 以下，all-to-all 负载恢复均衡。代价：略微增加 expert 选择的抖动（bias 调控更频繁），但对 loss 影响 < 0.1%
修改 ② — 增大 n_group / topk_group	将 router 的 n_group 从 8 增大到 16 ，topk_group 从 4 增大到 6 。这增加了 expert 选择的多样性——每个 token 从 16 个 group 中选 top-6 group，再从其中选 top-8 expert，减少了单一 expert 被“垄断”的概率。代价：路由计算量增加约 15%（router 是轻量操作，对总耗时影响 < 0.5%）
修改 ③ — Redundant Expert Deployment（推理可用，训练需评估）	对于检测到的高负载 expert（如 expert 193），在 EP group 内做冗余部署：将 expert 193 复制到 rank 17 和 rank 18 两张卡，token 随机分流到两张卡（各 50%）。但这对训练的一致性有影响（两条梯度的 expert weight 不同）——仅适用于推理场景，训练不推荐
修改 ④ — 训练侧兜底：增大 NCCL timeout	将 NCCL_IB_TIMEOUT 从 30s 增大到 60s ，防止极端过载时 all-to-all 超时触发 NCCL 错误导致训练中断。不解决根因但防止训练 crash

步骤	内容
验证	①+② 实施后：慢步占比从 10% 降至 < 2%，步耗时 CV 从 27% → 约 8%，all-to-all 耗时恢复到正常步 180~220ms 范围（慢步不再出现 >1s 的 all-to-all）。步耗时均值 10.3s → 约 10.0s（正常步小幅下降，因为减少了偶发的 all-to-all jitter 对流水线稳定性的影响）。MFU 均值 45% → 约 46%（慢步消除 + 流水线稳定性提升，但主要瓶颈仍在 PP bubble——见案例四）